

数据库应用接口实验报告

23307110043 孙天宇

一、实验目标

用自己熟悉的高级语言编写一个数据库初始化程序，将多种来源的外部数据导入MySql（或Oracle, SqlServer）

掌握高级语言操作数据库的方法，具体包括：

- 使用 Java 通过 JDBC 连接并操作 MySQL 数据库
- 实现外部数据（CSV格式）的读取和导入
- 掌握数据库表的动态创建和数据插入方法

二、实验环境

- 操作系统：Windows 10
- 数据库：MySQL 8.0
- 开发工具：VSCode
- 编程语言：Java（JDK 17）
- 数据库驱动：MySQL JDBC Driver (mysql-connector-j-9.2.0.jar)

三、实验步骤及实现

1. 数据库连接实现

使用 JDBC 实现了数据库连接类 DatabaseConnector.java：

```
public class DatabaseConnector {  
    private static final String URL = "jdbc:mysql://localhost:3306/db_lab";  
    private static final String USER = "root";  
    private static final String PASSWORD = "Tonysun050127";  
  
    public static Connection connect() {  
        try {  
            Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);  
            System.out.println("数据库连接成功! ");  
            return conn;  
        } catch (SQLException e) {  
            System.out.println("数据库连接失败: " + e.getMessage());  
            return null;  
        }  
    }  
}
```

2. 数据表创建实现

实现了 TableCreator.java 类来动态创建数据表：

```

public class TableCreator {
    public static void main(String[] args) {
        String[] createStatements = {
            "DROP TABLE IF EXISTS room" ,

            // 创建 student 表（保持不变）
            "CREATE TABLE IF NOT EXISTS student (" +
            "registno VARCHAR(20) PRIMARY KEY, " +
            "name VARCHAR(50), " +
            "kdno VARCHAR(10), " +
            "kcno VARCHAR(10), " +
            "ccno VARCHAR(10), " +
            "seat INT)",

            // 修正 room 表结构
            "CREATE TABLE IF NOT EXISTS room (" +
            "kdno VARCHAR(10), " +
            "kcno VARCHAR(10), " +
            "ccno VARCHAR(10), " +
            "kdname VARCHAR(100), " +
            "exptime DATETIME, " + // 修改为 DATETIME 类型
            "papename VARCHAR(100))"
        };

        try (Connection conn = DatabaseConnector.connect();
            Statement stmt = conn.createStatement()) {

            for (String sql : createStatements) {
                stmt.executeUpdate(sql);
            }
            System.out.println("数据表创建成功！");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

- 可以同时完成创建两个表格

3. CSV 数据读取实现

创建了 CSVReader.java 类来读取 CSV 格式的数据：

```

public class CSVReader {
    public static void main(String[] args) {
        String filePath = "student.csv"; // 可替换为 room.csv
        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            // 读取标题行
            String header = br.readLine();
            System.out.println("Header: " + header);

            while ((line = br.readLine()) != null) {
                // 处理带引号的CSV字段
                String[] values = line.split(",(?=(?:[^\"]*"|\"[^\"]*\")*\"[^\"]*$)", -1);

                // 移除字段两侧的引号
                for (int i = 0; i < values.length; i++) {
                    values[i] = values[i].replaceAll("^\"|\"$", "");
                }

                // 动态打印所有列
                StringBuilder output = new StringBuilder();
                for (String value : values) {
                    output.append(value).append(" | ");
                }
                System.out.println(output.toString());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

- 读取csv文件的操作，两个文件需修改局部代码分开进行

4. 数据导入实现

使用 DataInserter.java 类实现数据的导入：

```

public class DataInserter {
    // 配置项

    private static final String TARGET_TABLE = "room"; // 可改为 student
    private static final String CSV_FILE = "room.csv"; // 可改为 student.csv

    public static void main(String[] args) {
        try (Connection conn = DatabaseConnector.connect();
            BufferedReader br = new BufferedReader(new FileReader(CSV_FILE))) {

            // 清空目标表
            truncateTable(conn, TARGET_TABLE);

            // 读取标题行确定字段数
            String[] headers = br.readLine().split("(?=(?:[^\"]*" + "\"[^\"]*" + "\")*(\"[^\"]*" + "$)", -1);
            String sql = buildInsertSQL(TARGET_TABLE, headers.length);

            // 准备日期格式化器
            SimpleDateFormat[] dateFormats = {
                new SimpleDateFormat("yyyy-MM-dd H:mm"),
                new SimpleDateFormat("yyyy-MM-dd HH:mm")
            };

            Set<String> uniqueKeys = new HashSet<>();

            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
                String line;
                while ((line = br.readLine()) != null) {
                    String[] values = parseCSVLine(line, headers.length);

                    // 处理主键去重（根据表结构调整）
                    String uniqueKey = TARGET_TABLE.equals("room") ?
                        values[0] + values[1] + values[2] : // kdno+kcno+ccno
                        values[0]; // registno
                    if (!uniqueKeys.add(uniqueKey)) {
                        System.out.println("跳过重复记录: " + uniqueKey);
                        continue;
                    }

                    // 绑定参数
                    for (int i = 0; i < headers.length; i++) {
                        setPreparedStatementValue(pstmt, i+1, headers[i], values[i], dateFormats[i]);
                    }
                    pstmt.addBatch();
                }
            }
        }
    }
}

```

```

        }
        pstmt.executeBatch();
    }
    System.out.println("数据插入成功！插入表： " + TARGET_TABLE);
} catch (Exception e) {
    e.printStackTrace();
}
}

private static String[] parseCSVLine(String line, int expectedLength) {
    String[] values = line.split(",(?=(?:[^\"]*" + "[^\\"]*" + "[^\"]*" + "$)", -1);
    for (int i = 0; i < values.length; i++) {
        values[i] = values[i].replaceAll("^\"|\"$", "").trim();
        if (values[i].isEmpty()) values[i] = null;
    }
    return values.length == expectedLength ? values : new String[expectedLength];
}

private static void setPreparedStatementValue(PreparedStatement pstmt, int index,
    String columnName, String value, SimpleDateFormat[] dateFormats) throws SQLException {
    if (value == null) {
        pstmt.setNull(index, Types.VARCHAR);
        return;
    }
}

private static String buildInsertSQL(String tableName, int paramCount) {
    return "INSERT INTO " + tableName + " VALUES (" +
        String.join(",", java.util.Collections.nCopies(paramCount, "?")) + ")";
}

private static void truncateTable(Connection conn, String tableName) throws SQLException {
    try (Statement stmt = conn.createStatement()) {
        stmt.executeUpdate("TRUNCATE TABLE " + tableName);
        System.out.println("已清空表： " + tableName);
    }
}
}

```

- 插入两个csv文件的数据同样需要修改部分代码分开进行。这里注意了主键重复、日期格式等问题，保证room和student两个文件的信息均能正确导入到数据库中。

四、实验结果

1. 成功实现了数据库的连接
2. 成功创建了对应的数据表结构

```
● PS C:\Users\veget\Desktop\DB_Lab> c:; cd 'c:\Users\veget\Desktop\DP\cp_d3ritiu2cp3xs6oue4sqx9fld.argfile' 'TableCreator'
```

数据库连接成功！
数据表创建成功！

3. 成功读取了 CSV 格式的数据文件

0356303		区雪婷		1101		1		22		0	
0316002		林喜常		1101		1		23		0	
0316003		何枫易		1101		1		23		0	
0334024		郭颖凌		1101		1		23		0	
0334027		庄婕		1101		1		23		0	
0334028		李妍斐		1101		1		23		0	
0334029		赵婧		1101		1		23		0	
0334033		王斯司		1101		1		23		0	
0334074		黎明		1101		1		23		0	
0345024		杨柳		1101		1		23		0	
0345040		丁琪馥		1101		1		23		0	
0345045		顾玉娇		1101		1		23		0	
0345058		张苹		1101		1		23		0	
0350014		王斌鑫		1101		1		23		0	
0350015		刘晓莉		1101		1		23		0	
0350016		杨齐		1101		1		23		0	
0350022		李疑天		1101		1		23		0	
0350029		张爱芽		1101		1		23		0	
0350030		黄德婧		1101		1		23		0	
0350035		席瑛		1101		1		23		0	
0350036		周琳		1101		1		23		0	
0356025		赵夏阳		1101		1		23		0	
0356076		李豪杰		1101		1		23		0	
0356079		鞠小君		1101		1		23		0	
0356083		吴家强		1101		1		23		0	
0356101		李荣鑫		1101		1		23		0	
0356105		刘宇洲		1101		1		23		0	
0356113		黄晓栋		1101		1		23		0	
0356144		胡哲		1101		1		23		0	
0356148		黄晓勇		1101		1		23		0	
0356178		肖林		1101		1		23		0	
0356181		张琳		1101		1		23		0	


```
0356190 | 陈明宇 | 1101 | 1 | 23 | 0 |
0356248 | 黄惠媛 | 1101 | 1 | 23 | 0 |
0356308 | 郭施言 | 1101 | 1 | 23 | 0 |
0356312 | 赵蓉雅 | 1101 | 1 | 23 | 0 |
0356325 | 艾克拜尔安尼瓦尔 | 1101 | 1 | 23 | 0 |
0357054 | 杨松麟 | 1101 | 1 | 23 | 0 |
0357118 | 夏骏 | 1101 | 1 | 24 | 0 |
0357119 | 黄卓英 | 1101 | 1 | 24 | 0 |
0358100 | 刘昭 | 1101 | 1 | 24 | 0 |
PS C:\Users\veget\Desktop\DB_Lab>
```

4. 成功将 CSV 数据导入到数据库中

```
PS C:\Users\veget\Desktop\DB_Lab> c:; cd 'c:\Users\veget\Desktop\DB_Lab'; & 'D:\JDK\bin\java.exe' '@C:\U
p\cp_d3ritiu2cp3xs6oue4sqx9fld.argfile' 'DataInserter'
数据库连接成功!
已清空表: room
数据插入成功! 插入表: room
```

mysql_db	对象	room @db_lab (mysql_db) - 表				
db_lab	表配置文件	开始事务	单元格编辑器	筛选 & 排序	列	数据分析 工具
表						
room						
字段						
A-Z 索引						
外键						
检查						
触发器						
student						
字段						
A-Z 索引						
外键						
检查						
触发器						
user_profile						
字段						
A-Z 索引						
外键						
检查						
触发器						
user_record						
字段						
A-Z 索引						
外键						
检查						
触发器						
users						
视图						
函数						
查询						
information_schema						
表						
视图						
函数						
查询						
mysql						
performance_schema						
sys						
mysql_db						

kdno	kcno	ccno	kdname	exptime	papername
varchar	varchar	varchar	varchar(100)	datetime	varchar(100)
1101	1	1		2004-06-10 08:0	(Null)
1101	1	2		2004-06-10 08:2	(Null)
1101	1	3		2004-06-10 08:4	(Null)
1101	1	4		2004-06-10 09:0	(Null)
1101	1	5		2004-06-10 09:2	(Null)
1101	1	6		2004-06-10 09:5	(Null)
1101	1	7		2004-06-10 10:1	(Null)
1101	1	8		2004-06-10 10:3	(Null)
1101	1	9		2004-06-10 10:5	(Null)
1101	1	10		2004-06-10 11:1	(Null)
1101	1	11		2004-06-10 12:0	(Null)
1101	1	12		2004-06-10 12:2	(Null)
1101	1	13		2004-06-10 12:4	(Null)
1101	1	14		2004-06-10 13:0	(Null)
1101	1	15		2004-06-10 13:2	(Null)
1101	1	16		2004-06-10 13:4	(Null)
1101	1	17		2004-06-10 14:0	(Null)
1101	1	18		2004-06-10 14:2	(Null)
1101	1	19		2004-06-10 14:4	(Null)
1101	1	20		2004-06-10 15:2	(Null)
1101	1	21		2004-06-10 15:4	(Null)
1101	1	22		2004-06-10 16:0	(Null)
1101	1	23		2004-06-10 16:2	(Null)
1101	2	1		2004-06-10 08:0	(Null)
1101	2	2		2004-06-10 08:2	(Null)
1101	2	3		2004-06-10 08:4	(Null)
1101	2	4		2004-06-10 09:0	(Null)
1101	2	5		2004-06-10 09:2	(Null)
1101	2	6		2004-06-10 09:5	(Null)
1101	2	7		2004-06-10 10:1	(Null)

mysql_1

mysql_db

db_lab

表

room

字段

A-Z 索引

外键

检查

触发器

student

字段

A-Z 索引

外键

检查

触发器

user_profile

字段

A-Z 索引

外键

检查

触发器

user_record

字段

A-Z 索引

外键

检查

触发器

users

视图

函数

查询

information_schema

表

视图

函数

查询

mysql

表配置文件

开始事务

单元格编辑器

筛选 & 排序

列

数据分析

工

registno	name	kdno	kcno	ccno	SEAT
# int	varchar(50)	# int	# int	# int	# int
216001	罗仕通	1101	2	3	0
311091	刘其	1101	2	8	0
311093	陈云云	1101	2	8	0
311099	杨洋	1101	2	6	0
313002	吴智建	1101	2	4	0
313005	李刚	1101	2	4	0
313016	苏雪菲	1101	2	1	0
313019	刘亚娜	1101	2	4	0
313020	王梦竹	1101	2	4	0
313023	陈思	1101	2	10	0
313033	童开申	1101	2	9	0
313034	袁景毅	1101	2	9	0
313037	陈嘉进	1101	2	7	0
313040	胡伟	1101	2	10	0
313041	陈皓	1101	2	9	0
313042	葛佳	1101	2	9	0
313043	唐莉	1101	2	9	0
313044	李姿	1101	2	9	0
313048	金晶	1101	2	9	0
313056	李洁思	1101	2	9	0
313058	南明婕	1101	2	10	0
313063	周梦遐	1101	2	9	0
313064	陈甘露	1101	2	7	0
313065	衡洁	1101	2	10	0
313067	杨珊	1101	2	9	0
313068	易云	1101	2	7	0
313079	朱思悦	1101	2	9	0
313081	卢昱	1101	2	4	0
313083	陆俊卿	1101	2	4	0
313086	高飞	1101	2	4	0

五、思考

关于外部数据不完整或不一致的处理方法：

1. 数据完整性处理：
- 设置默认值：对于缺失的非空字段，可以设置合理的默认值
 - 数据验证：在导入前进行数据验证，对不符合要求的数据进行标记或拒绝导入
 - 日志记录：记录所有异常数据，便于后续人工处理
2. 数据一致性处理：
- 外键检查：在导入数据前检查外键关系

- 事务管理：使用数据库事务确保数据的一致性
- 数据清洗：对原始数据进行预处理，确保符合数据库的约束条件

处理原始数据的原则：

1. 数据验证优先：导入前进行数据格式和完整性验证
2. 异常处理完备：对所有可能的异常情况都要有处理措施
3. 日志记录详细：记录所有数据处理过程，便于追踪和调试
4. 事务管理严格：确保数据操作的原子性
5. 可回滚机制：在出现严重错误时能够回滚到操作前的状态

六、改进建议

1. 如何实现批量读取、导入数据（目前只能手工修改代码，一个一个文件分开操作）
2. 如何保证数据的安全性（例如密码应当加密后存入数据库）